ClearSign AI

Architecture and Design

Assignment 3

Course: Software Engineering

Due Date: 5th May 2026 | Total Marks: 100

Role	Name	Student ID
Team Member 1	Syed Zain Ali	BSCS24115
Team Member 2	Mubeen Butt	BSCS24063

1. Introduction

ClearSign AI is an AI-powered legal and financial document risk analysis platform designed to democratize access to document understanding. Its core purpose is to enable non-expert users — freelancers, loan applicants, tenants, patients, and general consumers — to identify, understand, and assess risk in complex legal and financial documents without requiring expensive professional consultations.

The platform provides five specialized analysis modules, each powered by a Large Language Model (LLM) accessed via the Groq API:

Module	Document Type	Core ML Function

Legal Contract Scanner	Employment, NDAs, service contracts	Red flag clause detection and classification
Privacy Policy Auditor	Terms of Service, Privacy Policies	Data risk summarization and scoring (A–F)
Loan Agreement Analyzer	Loan/credit agreements	Hidden cost extraction and benchmark comparison
Medical Consent Simplifier	Surgical/treatment consent forms	Plain-English translation of medical-legal jargon
Lease QA Chat	Commercial and residential leases	Conversational Q&A with clause citations

Technology Stack

Layer	Technology	Role
Presentation	HTML5 / CSS3 / Vanilla JS	Single-page application UI
Application	Python 3.12 / Flask 3.0	REST API routing and input validation
Service	Python modules (services/)	Business logic per analysis module
Infrastructure	Groq SDK + PyMuPDF	LLM API calls and PDF text extraction
Model	LLaMA 3.3 70B (via Groq)	Core NLP analysis engine

2. Architectural Design

2a. Chosen Architectural Style

ClearSign AI adopts a Layered (N-Tier) Architecture, specifically a four-layer structure composed of the Presentation Layer, Application Layer, Service Layer, and Infrastructure Layer.

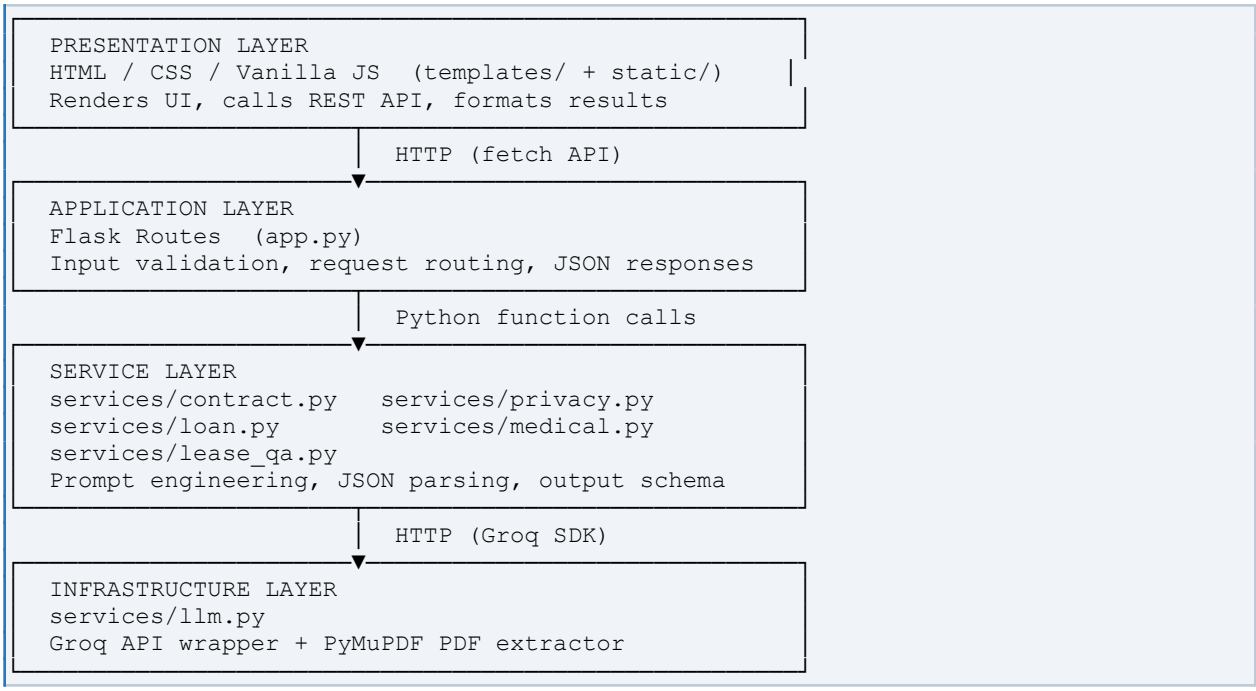
Justification

The Layered style was selected for the following reasons specific to this project:

- Separation of Concerns: The UI (HTML/CSS/JS), the web routing logic (Flask), the domain analysis logic (service modules), and the external API calls (Groq) are each confined to their own layer. A bug in the UI does not affect the analysis logic, and vice versa.
- Replaceability of the LLM Provider: All external API calls are isolated in services/llm.py. If the team switches from Groq to OpenAI or a local model, only that single file changes — no service module is touched.
- Independent Testability: Each service module (contract.py, privacy.py, etc.) is a plain Python function with no Flask dependency. It can be unit-tested in isolation by calling it directly with sample text.
- Simplicity for a Two-Person Team: Microservices would introduce deployment complexity (separate processes, inter-service HTTP calls, orchestration). Given the team size and academic context, Layered provides all benefits without operational overhead.

2b. Architectural Overview

Layer Diagram



Major Components and Their Responsibilities

Layer	Component	Technology
Presentation	Browser SPA	HTML/CSS/JS
Application	Flask Routes	Flask 3.0 (app.py)
Service	5 Analysis Modules	Python (services/)
Infrastructure	LLM + PDF Utilities	Groq SDK + PyMuPDF

Component Interaction — Request Flow

A typical user request flows as follows:

1. User pastes a contract in the browser and clicks "Scan for Red Flags".
2. JavaScript fetch() in app.js sends a POST request to /api/scan-contract.
3. app.py receives the request, validates input, and calls scan_contract(text) from services/contract.py.
4. contract.py builds a structured JSON-extraction prompt and calls call_llm(prompt) from services/llm.py.
5. llm.py sends the request to the Groq API and returns the raw JSON string.
6. contract.py parses the JSON and returns a normalised Python dict to app.py.
7. app.py serialises the dict to JSON and returns it as an HTTP response.
8. JavaScript renders the structured result as colour-coded risk cards in the browser.

Technology-to-Architecture Mapping

Architectural Role	Concrete Technology	Why This Choice
UI Framework	Vanilla JS (no framework)	Zero build step, minimal complexity for academic project
Web Server	Flask 3.0	Lightweight Python micro-framework; ideal for API-first apps
LLM Provider	Groq (LLaMA 3.3 70B)	Free tier, fast inference, JSON output mode
PDF Parsing	PyMuPDF (fitz)	Fastest Python PDF text extractor; handles multi-page docs
Prompt Engineering	Structured JSON prompts	Forces model output into parseable schema; reduces hallucination

3. Detailed Design

Three core modules are described in detail below. These represent the three primary NLP task types in the system: classification, scoring, and conversational Q&A.

3a. Module 1 — Legal Contract Scanner (services/contract.py)

Module Description

The Contract Scanner is the primary module of ClearSign AI. It receives raw contract text and returns a structured list of risky clauses, each with a severity classification, a plain-English explanation, and a negotiation tip.

Main functionalities:

- Clause segmentation — the LLM identifies discrete clauses within continuous text.
- Risk classification — each clause is labelled High, Medium, or Low severity.
- Explanation generation — converts legal jargon into plain English.
- Negotiation guidance — suggests concrete changes the signing party should request.
- Overall risk summarisation — a 2–3 sentence summary of the contract's risk profile.

Inputs, Outputs, and Dependencies:

Property	Details
Input	contract_text: str — raw text of the contract (up to 6,000 characters)
Output	dict: {overall_risk, summary, clauses: [{title, severity, excerpt, explanation, negotiation_tip}]}
Dependencies	services/llm.py — call_llm(), parse_json()

Called by	app.py → /api/scan-contract

Internal Structure — Pseudocode

```
FUNCTION scan_contract(text: str) → dict:

    prompt = BUILD_JSON_PROMPT(
        instruction = "Identify risky clauses. Return JSON with
                       overall_risk, summary, and clauses list.",
        contract_text = text[:6000]
    )

    raw_json = call_llm(prompt, max_tokens=4096)
    result   = parse_json(raw_json)

    IF "error" IN result:
        RETURN result          // propagate error to HTTP layer

    result.setdefault("overall_risk", "Unknown")
    result.setdefault("summary", "")
    result.setdefault("clauses", [])

    RETURN result
```

Output Schema

Key	Type	Description
overall_risk	str	"High" | "Medium" | "Low" | "Unknown"
summary	str	2–3 sentence plain-English risk overview
clauses	list	Zero or more clause objects
clauses[].title	str	Short name of the clause type
clauses[].severity	str	"High" | "Medium" | "Low"
clauses[].excerpt	str	Verbatim text snippet from the contract
clauses[].explanation	str	Why this clause is risky in plain English
clauses[].negotiation_tip	str	What the signing party should request to change

3b. Module 2 — Privacy Policy Auditor (services/privacy.py)

Module Description

The Privacy Auditor assigns a letter grade (A–F) to any privacy policy or Terms of Service document based on five evaluation criteria: volume and sensitivity of data collected, third-party sharing practices, user profiling and tracking, user rights and data deletion options, and transparency of language.

Main functionalities:

- Grade computation — synthesises five criteria into a single A–F score.
- Finding extraction — identifies specific invasive clauses with category labels.
- Severity ranking — labels each finding Critical, Important, or FYI.

- Plain-English verdict — 2-sentence summary of the policy's overall stance.

Inputs, Outputs, and Dependencies:

Property	Details
Input	policy_text: str — raw privacy policy or ToS text
Output	dict: {score, score_explanation, verdict, findings: [{category, severity, description, excerpt}]}
Dependencies	services/llm.py
Called by	app.py → /api/audit-privacy

Internal Structure — Pseudocode

```
FUNCTION audit_privacy(text: str) → dict:

    // Grading rubric embedded in prompt:
    // A = minimal data, no 3rd-party sharing, strong user rights
    // F = broad collection, sells data, no deletion option

    prompt = BUILD_JSON_PROMPT(
        rubric      = { A: "best practices", ..., F: "predatory" },
        policy_text = text[:6000]
    )

    raw_json = call_llm(prompt, max_tokens=3000)
    result   = parse_json(raw_json)

    result.setdefault("score", "?")
    result.setdefault("score_explanation", "")
    result.setdefault("verdict", "")
    result.setdefault("findings", [])

    RETURN result
```

Key Design Decision: The grading rubric is embedded directly inside the LLM prompt rather than computed programmatically. This exploits the LLM's text-reasoning capability for a classification task that would otherwise require a complex rule engine. The downside is non-determinism; the upside is that the rubric can be updated by editing a single string constant with no code changes.

3c. Module 3 — Lease QA Chat (services/lease_qa.py)

Module Description

The Lease QA module provides a stateless question-answering service over a lease document. It is the most architecturally distinct module because it involves two separate interactions: document loading (validation only) and question answering (retrieval + generation).

Main functionalities:

- Document validation — checks minimum word count and returns a size summary.
- Context-grounded answering — the LLM answers strictly from the provided lease text.
- Clause citation — the answer references the relevant lease section.

- Risk flagging — answers include a "Watch Out" note if a risk to the tenant exists.

Inputs, Outputs, and Dependencies:

Function	Input	Output
load_lease_text(text)	Lease text str	Status string (word/char count)
ask_lease_question(text, q)	Lease text + question str	Plain-text answer str

Stateless Design

Unlike a traditional chat application, lease_qa.py holds no state. The application layer (app.py) stores the lease text in an in-memory dict (_lease_store) and passes it to ask_lease_question() on every call. This keeps the service layer pure (no global state) and independently testable — you can call ask_lease_question(text, question) directly without any Flask context.

Internal Structure — Pseudocode

```
FUNCTION ask_lease_question(lease_text: str, question: str) → str:

    // QA uses plain-text mode (not JSON) for natural prose answers
    prompt = BUILD_TEXT_PROMPT(
        role_context = "Expert tenancy lawyer reviewing on behalf of tenant",
        instructions = [
            "Answer in max 300 words",
            "Cite the relevant clause or section",
            "Add 'Watch Out' note if a tenant risk exists",
            "If unanswerable from document, say so clearly"
        ],
        lease_text = lease_text[:8000],
        question   = question
    )

    answer = call_llm_text(prompt, max_tokens=600)

    RETURN answer    // plain string rendered in UI chat bubble
```

> Key Design Decision: This module uses call_llm_text() (plain-text mode) instead of call_llm() (JSON mode). A conversational answer is better expressed as flowing prose with clause citations than as a rigid JSON structure. The frontend renders it verbatim in a pre-formatted chat bubble.

4. Design Evaluation

4a. Modularity

ClearSign AI is divided into eight discrete units, each with a single, well-defined responsibility:

Unit	Responsibility
app.py	HTTP routing, request validation, JSON response serialisation
services/llm.py	All external I/O — Groq API calls and PyMuPDF PDF extraction
services/contract.py	Contract clause extraction and risk classification
services/privacy.py	Privacy policy grading and finding extraction
services/loan.py	Loan term extraction and benchmark comparison
services/medical.py	Medical jargon simplification and term flagging
services/lease_qa.py	Document validation and Q&A over lease text
static/js/app.js	UI state management and result rendering

Each service module has no import dependencies on any other service module. The only shared dependency is services/llm.py. Adding a sixth module (e.g., a Tax Return Analyzer) requires creating one new file (services/tax.py), adding one route in app.py, and adding one tab in the frontend. No existing code changes.

4b. Maintainability and Extensibility

- Changing the LLM model: Edit one line in services/llm.py (_MODEL = "..."). No other file requires changes.
- Adding a new document type: Create services/newmodule.py, add a route in app.py, add a tab in index.html. Estimated effort: 1–2 hours.
- Swapping the LLM provider: Replace the Groq SDK calls in services/llm.py with OpenAI or Anthropic SDK calls. The call_llm() and call_llm_text() function signatures remain unchanged, so all service modules are unaffected.
- Updating analysis criteria: Each prompt is a module-level string constant. The analysis criteria can be updated without touching Flask or JavaScript code.

4c. Scalability

Dimension	Current State	Scaling Path
Concurrent users	Limited by single Flask process	Deploy with Gunicorn (4–8 workers) or use async Flask
Request volume	Bounded by Groq free tier (~30 req/min)	Upgrade to paid tier or add Redis queue for buffering
Lease state	Stored in-process dict; lost on restart	Replace _lease_store with Redis; supports multi-worker
Feature expansion	Each new module = one file + one route	Architecture scales linearly; no refactoring required

4d. Trade-offs

- Simplicity vs. Determinism (LLM Dependency): Using a prompt-based LLM for all five modules is simpler than training task-specific models, but makes output non-deterministic. The same contract may receive slightly different risk classifications on two runs. Mitigation: temperature is set to 0.2 (near-greedy) to maximise consistency.
- Monolithic vs. Microservices: The five modules could be deployed as separate microservices, allowing independent scaling. However, the added complexity of service discovery, inter-service authentication, and container orchestration was not justified for a two-person academic project.
- In-Process State vs. Distributed Cache: The lease document is stored in a Python dict in memory. This is simple but breaks under multi-worker deployments. A Redis store would fix this but adds infrastructure complexity. The trade-off was accepted for the current scope.
- Vanilla JS vs. React: Using Jinja2 + Vanilla JS avoids a build toolchain (Webpack/Vite). The trade-off is less component reusability in the frontend, but this is acceptable given the small number of interactive elements in the UI.

4e. Alternative Designs Considered

Alternative 1: Microservices Architecture

Each of the five analysis modules could be deployed as an independent service (e.g., five separate FastAPI processes, each as a Docker container). This was rejected because:

- It requires container orchestration (Kubernetes or Docker Compose), adding significant operational complexity.
- Inter-service communication overhead is unnecessary when all modules share the same LLM provider.
- Team size (2 people) does not justify the operational burden during an academic semester.

Alternative 2: React / Next.js Frontend

A React frontend would provide better component reusability and a richer developer experience. It was rejected because:

- It requires a Node.js build pipeline (Webpack/Vite) and npm dependency management, adding setup complexity.
- The UI is relatively simple (5 tabs, each with a textarea and output area) and does not benefit significantly from a component framework.
- Vanilla JS achieves the same result without any build tooling, keeping the project entirely self-contained.

Alternative 3: Task-Specific Fine-Tuned Models (Hugging Face)

Each module could use a fine-tuned model (e.g., a BERT model trained on legal contracts for clause classification). This was rejected because:

- No publicly available labeled datasets exist for all five document types in the project scope.
- Fine-tuning requires GPU resources not available to the team.
- A general-purpose LLM (LLaMA 3.3 70B) achieves comparable quality through prompt engineering alone, making fine-tuning unnecessary.